

TP2 : Casser des codes secrets avec classe

Dans ce TP, vous mettrez en place le système de chiffrement par substitution, et comment "casser" un cryptage réalisé avec ce système. Vous implémenterez les méthodes demandées à l'aide de classes, ce qui vous permettra de faire votre premier contact avec les classes en Python.

1 Présentation du codage par substitution (extrait du TP d'Informatique de base - groupe avancé) (5mn)

Le chiffrement par substitution consiste à remplacer, dans un message, chaque lettre par une autre lettre. Deux lettres égales dans le texte original doivent être chiffrées de la même manière, et deux lettres différentes doivent être chiffrées différemment. C'est comme si on prenait les lettres de l'alphabet, rangées par ordre croissant, qu'on les mélangeait (obtenant alors un nouvel alphabet) et qu'on utilisait le nouvel alphabet pour chiffrer notre message : les "a" de notre message deviendraient alors la première lettre de notre nouvel alphabet, etc...

Par exemple, considérons cette substitution :

```
lettre originale : a b c d e f g h i j k l m n o p q r s t u v w x y z
mot de passe    : w q a x s z c d e v f r b g t n h y j u k i l o m p
```

Les 'a' du texte original deviendront des 'w' dans le texte chiffré, les 'b' deviendront des 'q', etc. Le message "La programmation, c'est vraiment bien !" serait ainsi transformé :

```
l a p r o g r a m m a t i o n , c ' e s t v r a i m e n t b i e n !
r w n y t c y w b b w u e t g , a ' s j u i y w e b s g u q e s g !
```

Remarquez que le chiffrement de César (où chaque lettre est décalée d'un même nombre de cran dans l'alphabet) est un cas particulier de chiffrement par substitution. Remarquez aussi que ce nous ne prendrons pas en compte les lettres accentuées ni les signes de ponctuation dans notre système de cryptage.

2 Création d'une classe codage permettant de réaliser le chiffrement d'un texte (45mn)

Questions

1. Dans un premier temps, déclarez une classe **Encodeur**, qui n'hérite d'aucune autre classe, à l'aide du code suivant :

Code Python

```
class Encodeur:
    pass
```

Le mot clef **pass** en Python est utilisé partout où du code est requis, mais qu'aucun code n'est marqué. Ici, pour le moment, nous n'écrivons rien dans la classe **Encodeur**, donc nous pouvons marquer ce mot clef temporairement.

2. Nous allons proposer un constructeur pour notre classe. Le constructeur d'une classe s'appelle forcément **__init__** (remarquez les deux underscores de part et d'autre du mot clef), et prend au moins comme argument le mot clef **self** (qui représente l'instance de classe).

Il est important de noter que toute méthode de classe non statique en Python prend comme premier paramètre **self**.

Notre constructeur prendra comme paramètre une chaîne de caractères **pwd**, représentant le mot de passe que l'on utilisera pour le cryptage. Ce mot de passe sera donc un alphabet désordonné (26 lettres minuscules, toutes différentes). Le mot de passe sera conservé dans une variable de classe nommée **password**.

Nous écrirons donc :

Code Python

```
class Encodeur:
    def __init__(self, pwd):
        self.password = list(pwd)
```

Remarquez que, contrairement à Java, les variables de classe ne sont pas, en Python, déclarées en préambule de la classe. Elles sont déclarées lorsqu'elles sont nécessaires (en général, dans le constructeur) grâce au mot clef **self.nom_variable**. Testez votre code en construisant un objet **Encodeur** et en vérifiant le contenu du mot de passe :

Code Python

```
enc = Encodeur("bcdefghijklmnopqrstuvwxyz")
print(enc.password)
```

3. Nous souhaitons donner la possibilité à l'utilisateur, lorsqu'il construit un objet **Encodeur**, de ne pas spécifier un mot de passe. Dans ce cas, la classe devra générer elle-même un mot de passe. Dans ce cas, nous commençons par spécifier que le paramètre **pwd** peut avoir une valeur par défaut (qui sera nulle), impliquant qu'il ne sera pas forcément nécessaire de lui spécifier une valeur lors de l'appel de la fonction :

Code Python

```
class Encodeur:
    def __init__(self, pwd=None):
        self.password = list(pwd)
```

Vous devrez maintenant modifier le constructeur pour que ce dernier analyse la valeur de **pwd**, grâce au test

Code Python

```
if pwd is None:
```

Si le mot de passe n'a pas été spécifié, vous devrez en générer un. Vous pouvez, grâce à la commande suivante, générer dans la variable **self.password** un alphabet ordonné sous forme de chaîne de caractères :

Code Python

```
import string

self.password = string.ascii_lowercase
```

Ensuite, regardez la méthode **shuffle** de la classe **random**. Testez ensuite votre code et vérifiez que, dans le cas d'un appel au constructeur sans paramètre, un **Encodeur** avec un mot de passe aléatoire est bien créé.

4. Nous souhaitons proposer une méthode de classe `encode_string(self, input_string)` qui permettra de renvoyer la chaîne de caractères donnée en entrée, codée à l'aide de la méthode de substitution et du mot de passe de la classe.

Défi : Vous devriez pouvoir, en une seule ligne de code, à l'aide d'une compréhension de liste, générer une liste de caractères représentant la chaîne d'entrée cryptée.

Nous vous proposons de réfléchir à ces étapes pour vous aider à trouver la bonne manière de faire :

- Parcourez la chaîne de caractères, et placez dans une nouvelle liste uniquement les caractères minuscules à l'aide de la méthode `islower()` de la classe `string` (utilisez la compréhension de liste pour cela).
- La valeur cryptée de chaque caractère de cette nouvelle liste dépendra de sa position dans l'alphabet : vous pouvez extraire cette position en soustrayant, à chaque caractère, le code `ascii` de la lettre `a` minuscule. Ainsi, si on possède un caractère `c` représentant une lettre minuscule, on pourra extraire sa position dans l'alphabet à l'aide de

Code Python

```
position = ord(c) - ord('a')
```

Important : Vous trouverez une autre stratégie, sur Internet, pour récupérer la position d'une lettre minuscule dans l'alphabet. Celle-ci propose de faire :

Code Python

```
position = string.ascii_lowercase.index(c)
```

Comprenez-vous ce que fait ce code ? Que pensez-vous de cette stratégie ?

- Maintenant que l'on connaît la position du caractère dans l'alphabet, on peut aisément trouver la lettre cryptée lui correspondant :

Code Python

```
crypt = self.password[position]
```

Inspirez-vous de ces étapes pour construire la liste des lettres du message crypté en une seule ligne de code. Il vous faudra, dans votre solution, encoder les minuscules en minuscules, les majuscules en majuscules, et laisser tels quels les caractères non lettre (et le tout en une seule ligne de code). Pensez ensuite à transformer votre liste en une chaîne de caractères en utilisant la fonction `join`.

5. Proposez une méthode de classe `encode_file(self, input_file, output_file)` permettant d'encoder tout un fichier d'entrée vers un fichier de sortie. Vos fonctions devront prendre en compte l'exception `IOError` et la gérer en cas de fichier non trouvé :

Code Python

```
try:
    #On lit le fichier en entree
    text_file = open(input_file, 'r')
    #...faire autre chose si nécessaire
except IOError:
    print("Cannot open file "+input_file)
    return
```

Pour la suite de cette méthode, essayez de lire l'intégralité du fichier dans une chaîne de caractères, ce qui vous permettra d'appeler la méthode précédente pour réaliser le cryptage.

3 Création d'une classe de décodage permettant de réaliser le déchiffrement d'un texte (30mn)

Dans la suite, vous créerez une classe de décodage permettant de décoder un texte à partir du mot de passe ayant permis le chiffrement de ce dernier.

L'idée centrale ici est, que pour réaliser le décodage d'un texte, il suffit de recoder le texte chiffré avec le mot de passe "inverse" de celui utilisé pour le codage. Si toutes les lettres 'a' avaient été transformées en 'u', il faudra, dans le décodage, que toutes les lettres 'u' soient transformées en 'a'.

Pour construire le mot de passe "inverse", il faudra donc que, si la première lettre du mot de passe de cryptage était 'u' (21ème lettre de l'alphabet), la 21ème lettre du mot de passe inverse soit 'a'. Essayez, sur papier, d'écrire un mot de passe au hasard, puis de construire le mot de passe inverse.

Questions

1. Construisez une classe **Decodeur** avec un constructeur prenant en paramètre le mot de passe de codage que l'on souhaite inverser. Votre constructeur inversera le mot de passe, et initialisera un objet **Encodeur** avec ce mot de passe inverse.

Pour ce faire, commencez par construire des paires d'éléments à l'aide de **zip** : chaque lettre du mot de passe de cryptage doit être associée avec la lettre, dans l'alphabet, qui correspond à sa position dans le mot de passe. Par exemple, si le mot de passe de cryptage commence par "uod...", alors le 'u' sera associé avec 'a' (première lettre), 'o' sera associé avec 'b', 'd' sera associé avec 'c', etc.

A partir de là, à vous de trouver ce qu'il faut faire pour inverser le mot de passe, mais rappelez-vous que, d'après notre exemple, la lettre 'a' devra être placée à la 21ème position de notre mot de passe de décryptage. Un tri basé sur les premières lettres de chaque pair permettrait naturellement d'ammener le 'a' associé avec le 'u' à la 21ème position...

2. Proposez deux méthodes de classe **decode_string** et **decode_file** réalisant le décodage d'une chaîne de caractères et le décodage d'un fichier. Ces méthodes feront appel aux méthodes correspondantes de la classe **Encodeur**.

4 Casser le mot de passe d'un texte

4.1 Description générale de la méthode (5mn)

Nous nous retrouvons maintenant dans la situation où un texte est crypté avec un mot de passe, mais on ne connaît pas ce dernier... On considère que le texte crypté était en anglais (ainsi, pas de problème avec les accents).

On souhaite faire un programme qui testera des mots de passe "probables" afin que le texte décrypté ressemble à de l'anglais.

Cependant, comment estimer si un texte ressemble à de l'anglais? On possède un fichier *quadgram_en.txt* qui rassemble une très grande partie des combinaisons (389374 sur 456976) de quatre lettres en anglais, avec un score de popularité à côté. Chaque quadgram est plus ou moins populaire selon la langue utilisée (par exemple, en français, TION est un quadgram très utilisé car il apparaît souvent).

Pour calculer à quel point une phrase est anglaise, on décomposera cette dernière en quadgram, puis on calculera le score de chaque quadgram. Par exemple, pour calculer le score de "How are you?", il faudra aller chercher la valeur du quadgram "howa", celle du quadgram "owar", celle de "ware", de "arey", de "reyo" et de "eyou", et additionner le log de toutes les valeurs récupérées.

Pour déchiffrer le texte, il faut trouver le bon mot de passe (qui est l'alphabet mélangé qui annulera les effets du cryptage effectué)... Pour ce faire, commencez par générer le mot de passe "abcdefgh....xyz" (c'est à dire que la lettre a est remplacée par a, b par b, etc... bref, rien ne change) et calculez le score du texte complet (en mesurant le score de chaque quadgram).

Ensuite, permutuez au hasard deux lettres de votre mot de passe, et refaîtes le calcul du score : si ce dernier est meilleur que le précédent, conservez le nouveau mot de passe ; sinon, revenez au précédent. Répétez ce processus jusqu'à ce qu'aucune permutation ne permette d'améliorer le score (par exemple, si après 1000 permutations vous n'avez pas réussi à améliorer le score, considérez que vous pouvez arrêter la boucle). Vous aurez alors normalement le bon mot de passe qui vous permettra de décrypter le texte.

Cette méthode s'appelle une descente de gradient, et nous allons maintenant voir comment la coder en Python.

4.2 Stocker les quadgrams dans une structure (25mn)

Questions

Proposez une classe **CodeBreaker** dont le constructeur prendra en paramètre un fichier quadgram.

En vous inspirant du code de la semaine dernière, lisez le fichier de quadgram dans le constructeur comme s'il s'agissait d'un fichier csv, et récupérez les paires d'éléments (quadgram, score). Dans quelle structure stockeriez-vous les valeurs lues ? Rappelez-vous qu'il faudra, à l'avenir, être capable de facilement trouver le score d'un quadgram spécifique. On recherchera donc des valeurs de score par rapport à une clef étant le quadgram.

4.3 Attaquer le code par permutation (35mn)

Questions

1. Proposez, dans votre classe, une méthode prenant en paramètre une chaîne de caractères et renvoyant une nouvelle chaîne constituée uniquement des lettres de la première chaîne, avec les majuscules transformées en minuscules, sans aucun signe de ponctuation ou autre. Une seule ligne de code suffira à résoudre ce problème (considérez la chaîne de caractères comme une liste, écrivez une compréhension de liste pour effectuer la tâche, et regardez la fonction **join** de la classe string).
2. Proposez, dans votre classe, une méthode prenant en paramètre un alphabet désordonné en paramètre (un mot de passe pour le codage par substitution), et deux positions i et j dans cet alphabet, dont la valeur par défaut sera *None*. Si i et j ont une valeur, on échange les lettres à la position i et j et on renvoie le nouvel alphabet. Si i et j n'ont pas de valeur, on les choisit au hasard puis on réalise l'échange. On renvoie le nouvel alphabet ainsi que la valeur de i et j .
3. Proposez, dans votre classe, une méthode prenant en paramètre une chaîne de caractères et renvoyant le score de cette chaîne basé sur le fichier quadgram avec lequel la classe avait été construite. On considérera, pour des soucis de performances, que la chaîne ne contient que des lettres minuscules (aucun signe de ponctuation, aucun chiffre, les majuscules sont minuscules). On rappelle que le score d'une chaîne est la somme des log des scores des quadgrams constituant cette chaîne.

Afin de réaliser cette tâche, commencez par construire une liste de chaque groupe de 4 lettres (quadgram) présent dans votre texte. Ensuite, à l'aide d'une compréhension de liste, récupérez les scores associés à chaque groupe quadgram ainsi formé.

Pour vous aider dans votre travail, le score du texte **mystery_text.txt** transformé par la fonction précédente (majuscule en minuscule et pas de ponctuation) est de 12176.13.

4. Proposez, dans votre classe, une méthode prenant une chaîne de caractères cryptée et qui tentera de casser le code de cryptage. Votre méthode commencera par retirer, de la chaîne de caractères, toutes les non lettres et transformera les majuscules en minuscules.

Ensuite, vous construirez un objet **Encodeur** initialisé avec un mot de passe égal à l'alphabet. Vous pourrez ensuite écrire la boucle principale :

- (a) Calculer le score du texte décrypté par le mot de passe en cours (au début, le mot de passe est égal à l'alphabet).
- (b) Permuter deux lettres au hasard du mot de passe en cours, et décrypter le texte avec ce nouveau mot de passe.
- (c) Calculer le score du texte décrypté :
 - Si ce texte a un meilleur score que le score calculé à la première étape, le nouveau mot de passe devient le mot de passe en cours.
 - Sinon, on annule la permutation pour revenir au mot de passe d'avant.
- (d) Si on a tenté 1000 permutations de mots de passe d'affilée sans jamais améliorer le score, on arrête et on affiche le texte décrypté. Sinon, on recommence à l'étape 2.

Évitez, dans votre implémentation, de créer un nouvel Encodeur à chaque tour de boucle, mais essayez plutôt de modifier directement le mot de passe associé à l'encodeur.

Pensez, une fois le mot de passe trouvé, à effectuer un décryptage du fichier d'origine avec ce mot de passe, afin de reconstituer le bon format de fichier (espaces, onctuations,

majuscules,...) dans le fichier de sortie.

Testez votre code sur le fichier `mystery_text.txt`.

5 Rendu du programme

Vous devrez ensuite rendre tous vos fichiers python, **sauf le dossier `venv`**, dans un fichier zip dont le nom aura comme format

Code Python

```
VotreNom_VotrePrenom_Votre_NomeroEtudiant.zip
```

sans espace ni accent. Si vous avez un binôme, il vous faudra écrire son nom, prénom et numéro d'étudiant séparés par des espaces, en plus des vôtres, dans un fichier `binome.txt`, dont le format sera

Code Python

```
Nom1 Prenom1 Numero_etudiant1  
Nom2 Prenom2 Numero_etudiant2
```

Vous ne devez rendre le fichier qu'une seule fois par binôme.

Vous devrez rendre trois programmes permettant de respectivement faire l'encodage d'un texte à l'aide d'un mot de passe, le décodage d'une texte à l'aide d'un mot de passe, et casser le code d'un texte. Vos programmes devront s'exécuter à l'aide des commandes :

Code Python

```
python3 encodage.py fichier_entree.txt mot_de_passe fichier_sortie.txt
```

où `fichier_entree.txt` est le fichier contenant le texte à encoder, `fichier_sortie.txt` contiendra le texte crypté et `mot_de_passe` sera le mot de passe

Code Python

```
python3 decodage.py fichier_entree.txt mot_de_passe fichier_sortie.txt
```

où `fichier_entree.txt` est le fichier contenant le texte à décoder, `fichier_sortie.txt` contiendra le texte décrypté et `mot_de_passe` sera le mot de passe précédemment utilisé pour l'encryptage

Code Python

```
python3 cassercode.py fichier_entree.txt fichier_quadgram.txt fichier_sortie.txt
```

où `fichier_entree.txt` contiendra le message à décrypter, et `fichier_sortie.txt` contiendra le résultat du décryptage.

5.1 JARVIS

Pour vous aider dans votre travail, vous pouvez utiliser JARVIS. Pour tester immédiatement votre programme, préparez votre fichier comme expliqué ci-dessus. Puis, envoyez le à l'adresse :

```
chaussard.laga+pytp2_encod@gmail.com
```

pour le travail sur l'encodeur

```
chaussard.laga+pytp2_decod@gmail.com
```

pour le travail sur le décodeur

`chaussard.laga+pytp2_casser@gmail.com`

pour le travail sur le cassage du code.

Deux minutes plus tard, vous devriez recevoir un mail de JARVIS qui aura testé votre programme et vous transmettra les résultats de ses tests. Votre message n'a pas besoin d'avoir de sujet particulier ou bien de message particulier, simplement votre programme en fichier joint. N'incluez qu'un seul fichier avec votre mail (un fichier compressé selon les instructions précédentes).

6 Facultatif - Gérer les mots de passe invalides dans la classe Decodage

On souhaite que le constructeur de la classe **Encodeur** puisse générer des exceptions dans le cas où le mot de passe utilisé pour initialiser la classe serait invalide.

On souhaite pouvoir générer plusieurs types d'exceptions selon les différents types d'erreurs pouvant survenir :

- Une exception pourra survenir si le mot de passe n'a pas la bonne longueur (26 lettres).
- Une exception pourra survenir si le mot de passe n'est pas uniquement constitué de lettres minuscules.
- Une exception pourra survenir si une lettre minuscule apparaît deux fois dans le mot de passe.

On souhaite construire des classes pour chacune des ces exceptions, et modifier le constructeur de **Encodeur** afin qu'il génère la bonne exception dans le cas où le mot de passe ne serait pas valide. Vos classes devront incorporer, dans leur constructeur, un message explicite sur l'erreur survenue qui s'affichera automatiquement lorsque l'exception surviendra.

Cependant, dans notre code principal, comme nous sommes un peu fatigués par un TP si long, on ne souhaite pas tester toutes ces exceptions... Comment faire pour n'avoir à tester qu'une seule exception et capturer n'importe laquelle des exceptions pouvant être générée par le constructeur ?